

Week 5

Game Engine Development Unity

VFX

Hooman Salamat

# Objective & Concept

## Objective

- Define the acronym VFX
- Explain different applications of VFX in real-time 3D experiences, such as gameplay and environmental effects
- Describe the impact that VFX can have on the level of polish in a project
- Understand the differences between Unity's Built-in Particle System and VFX Graph

## Concepts

- VFX are simulated motion effects added to enhance a scene, ranging from a subtle splash of water to a massive fiery explosion. In this lecture, you will learn what VFX are, who makes them in the industry, and then you will tinker with a fire effect in Unity.

# VFX overview

---

**Visual Effects (VFX)** can be the wow factor in a real-time 3D project, ranging from a subtle splash of water to a massive fiery explosion.

Without VFX, an environment can feel dull. These effects introduce movement into an environment, bringing it to life.



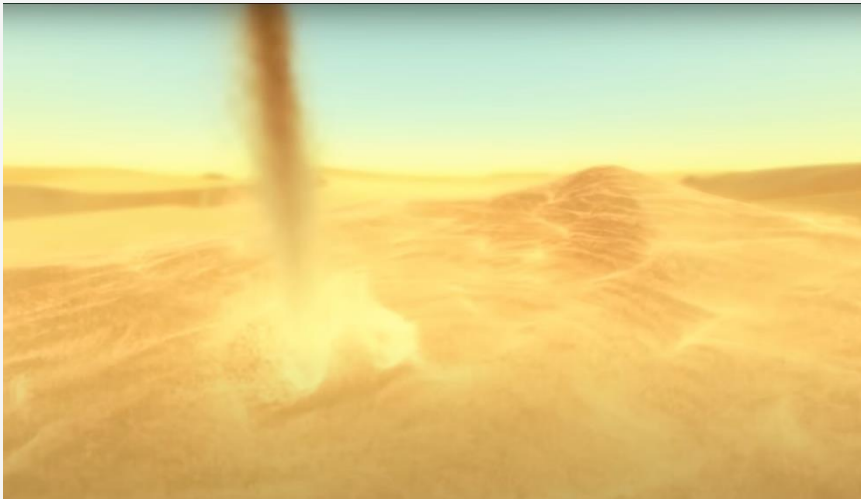
# What are VFX?

---

VFX can be broken down into two categories:

**Environmental effects** are meant to represent physical occurrences, such as fire, rain, mist, or explosions.

**Gameplay effects** are meant to enhance or draw attention to specific user interactions. For example, you could add a glowing effect when an object is selected, or a celebration effect that accompanies an in-game achievement.



# Identify the VFX

---

The first image shows an environmental mist effect hovering over the water. This adds an element of mystery and danger to the scene.

The second image shows a gameplay effect, where a blue slash trail follows the player's swinging staff. This makes the swinging interaction much more satisfying and noticeable for the player.



# What does a VFX artist do?

---

A VFX artist is usually responsible for any **computer-generated imagery (CGI)** that moves but isn't a character or animal.

If it doesn't move (like a tree or building), it's probably made by a 3D artist, including 3D modelers and texture artists.

If it's an animated character, it's probably the responsibility of a character artist, rigging artist, or animator.

When people think of VFX, they usually think of live action movies, where effects are added on top of real footage.

Most VFX artists will have a foundation in traditional art principles, like color, texture, and composition. They will also be familiar with physics and fluid dynamics in order to understand how liquids, types of cloth, or particles behave in different conditions.

# Tools

---

In addition to Unity, VFX artists might use or be familiar with other software programs, including the following:

2D image editing software, such as [Photoshop](#), to create 2D textures and particles

3D modeling and animation software, such as [Maya](#), to create 3D shapes for certain effects

Software specializing in procedural simulations like fluids and particles, such as [Houdini](#)

# Particle Systems vs VFX Graph

---

Unity actually has two separate systems for developing VFX: the Built-in Particle System and VFX Graph.

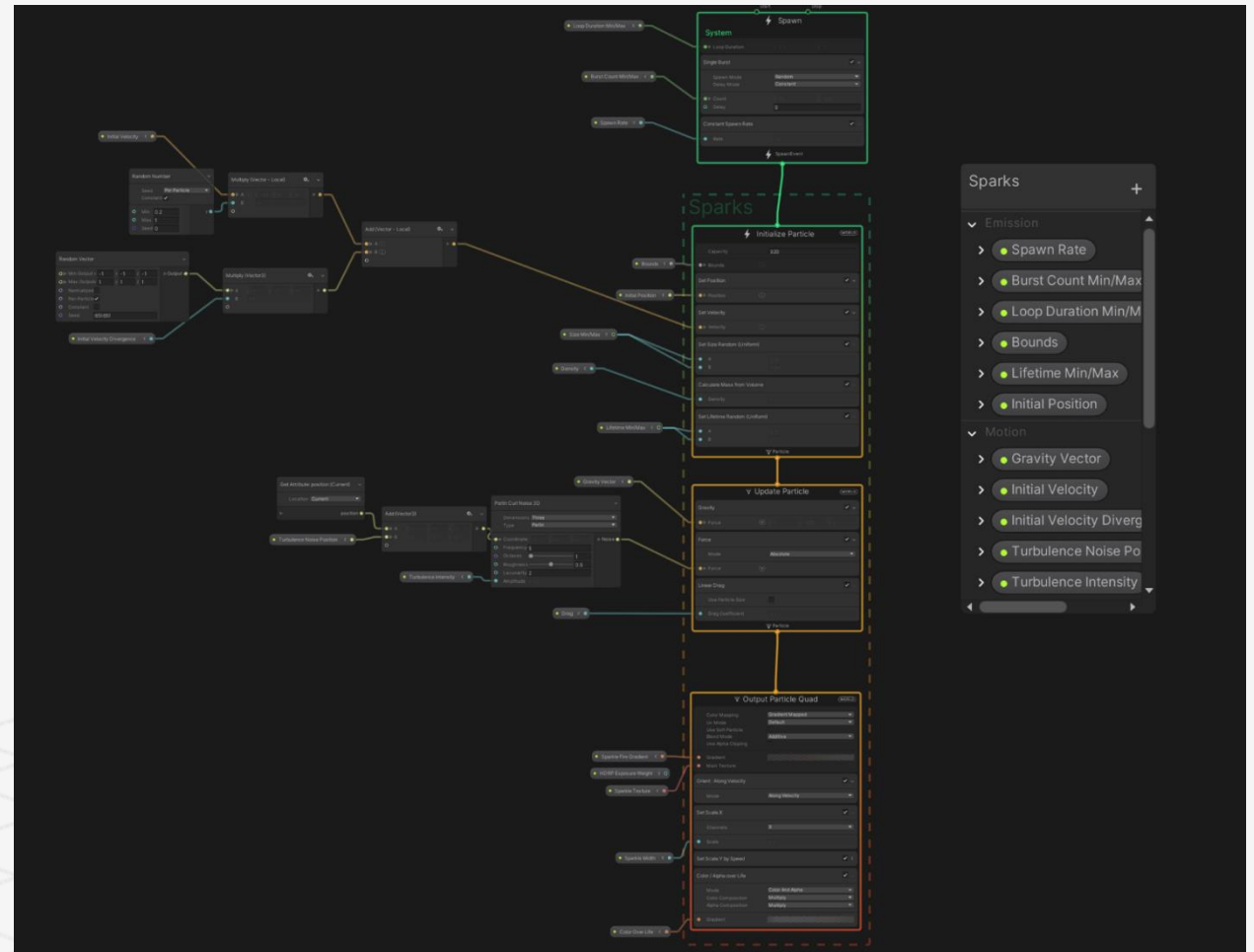
The Built-in Particle System is Unity's default component-based system that you will find in most Unity projects. VFX Graph is a newer, node-based system, which is extremely powerful but a bit more difficult to learn.

Here are examples of some highly complex simulations made possible with VFX Graph.



# VFX Graph

VFX Graph is a node-based editor for creating effects in Unity. Here is an example of what a VFX Graph might look like for a more complex effect:



# Advantages vs. Disadvantages

---

## Advantages of VFX Graph

VFX Graph can process millions of simultaneous particles, while Particle Systems can only process thousands.

VFX Graph can produce more complex simulations, such as the genie shown above, which would not be feasible using a Particle System.

## Disadvantages of VFX Graph

VFX Graph is a bit more challenging for beginners to learn than Particle Systems.

VFX Graph has stricter [hardware requirements](#) than Particle Systems, preventing it from running successfully on certain target devices.

Since VFX Graph runs on the GPU and Unity physics calculations are done on the CPU, VFX Graph particles cannot interact with the rest of your app's physics (collide with the ground, for example). Particle Systems run on the CPU, so they can interact with other physics-based components.

# Choosing between VFX Graph and Particle Systems

---

How do you choose whether to use VFX Graph or the Particle System?

If you're pretty experienced and want to create something really complex or something that requires millions of particles, you might want to use VFX Graph.

If you're not an expert and want to create something simple, something that runs on all devices, or something that can interact with other physics-based objects in your scene, use a Particle System.

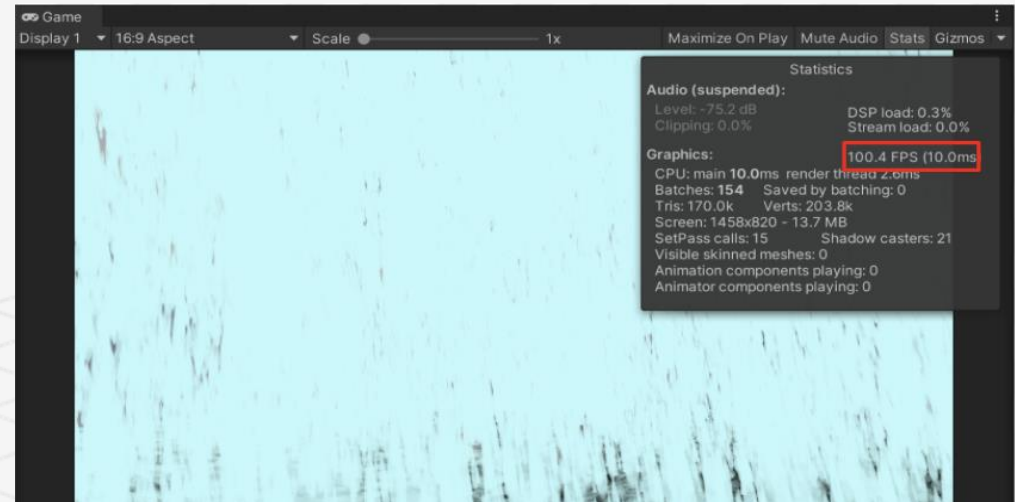
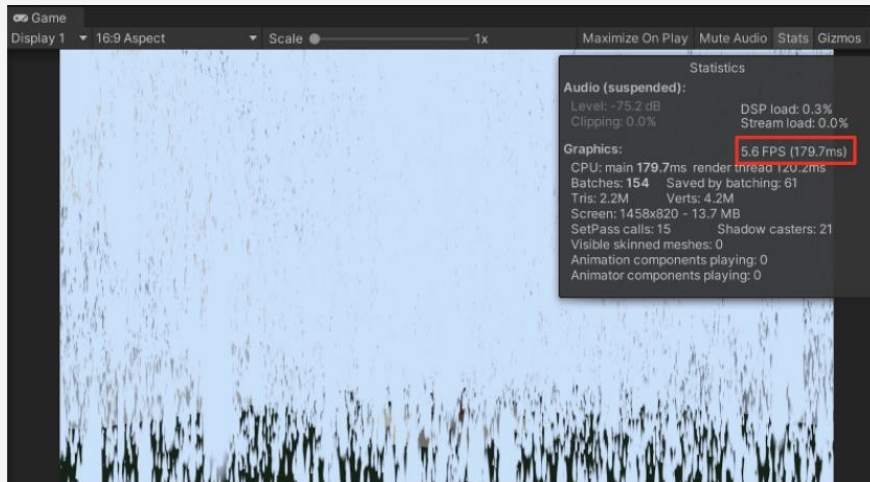


# Example

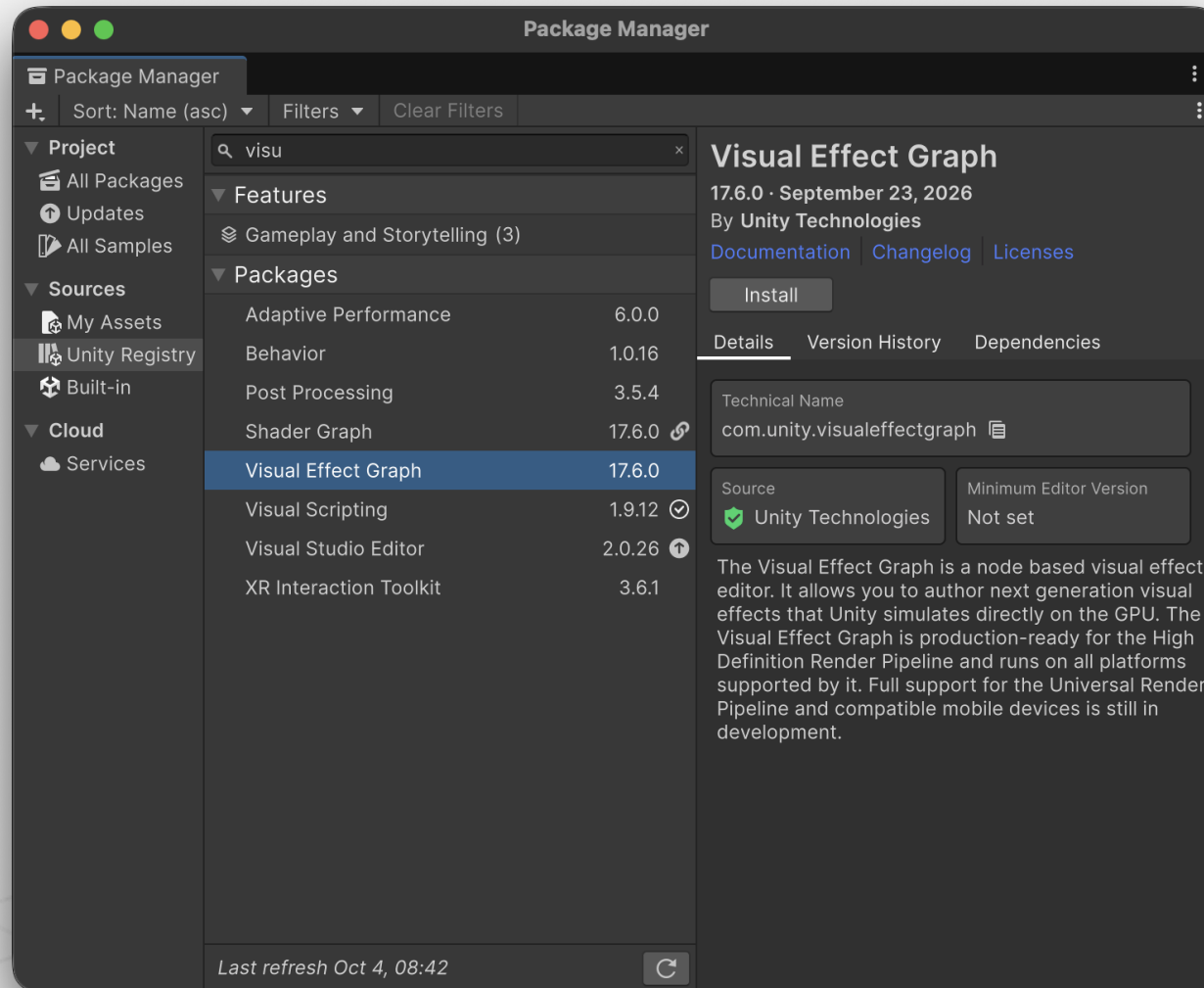
In the case of the weather effect in the scene, we may want a higher number of particles.

Take a look at the two screenshots. The first shows a Particle System simulating one million particles. It has an abysmal frame rate of around 5fps.

Compare that to this effect, which uses VFX Graph to simulate one million particles, and still has an excellent frame rate of around 100 frames per second.

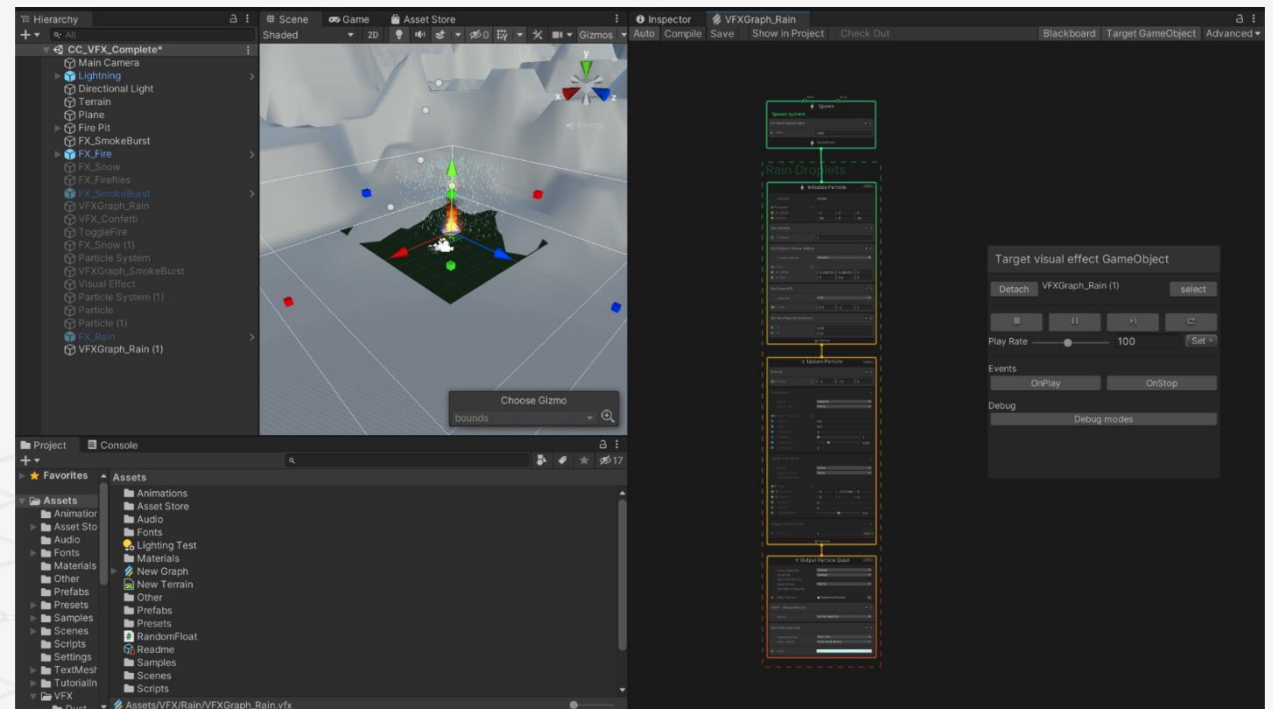
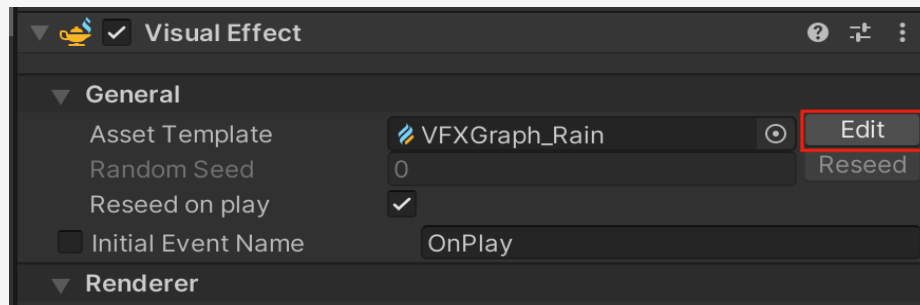


# Visual Effect Graph



# Edit Visual Effect Graph

1. With the VFX Graph object selected, locate the **Visual Effect** component. Notice the little lamp icon next to the component's name – that is the icon for VFX Graph.
2. Select the **Edit** button to open the VFX Graph editor window.



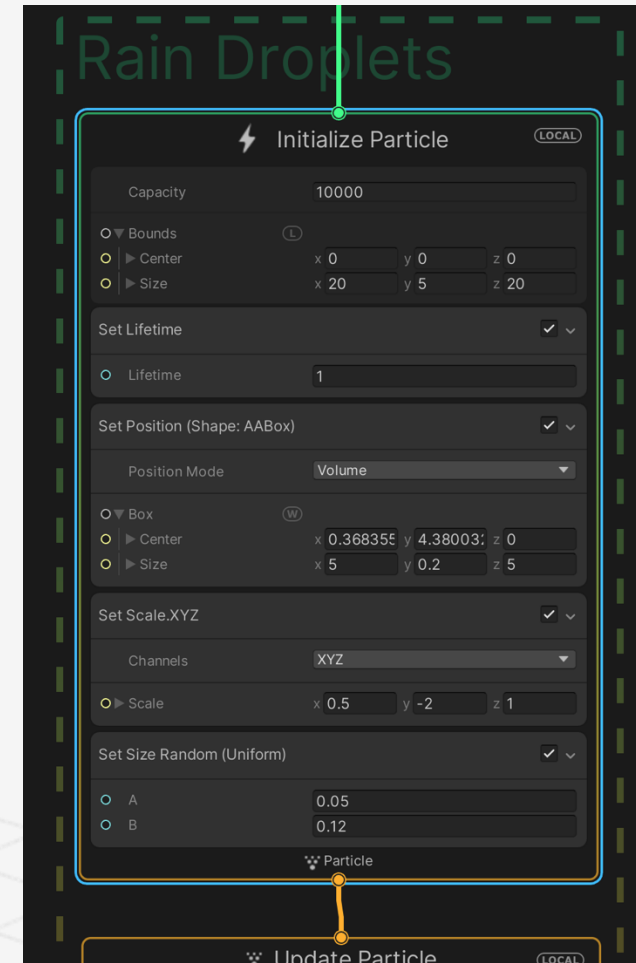
# Navigate the VFX Graph window

**Pan:** Middle-click and drag, or hold **Alt** (Windows) or **Option** (macOS) and click and drag.

**Zoom:** Rotate the wheel or use your trackpad zoom.

**Focus and zoom in:** Select an element and press the **F** key.

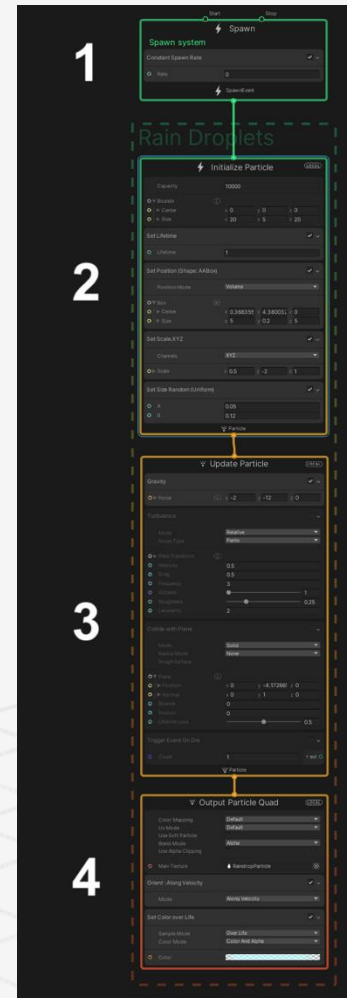
**Fit to window:** Press the **A** key.



# Nodes

With the full VFX Graph in view, the first thing you'll notice is that it is made up of four main nodes, which are stacked vertically on top of each other.

These four default nodes are called the **Context Nodes** (or “Contexts” for short) and are read from top to bottom. We'll go through those in detail in the next step.



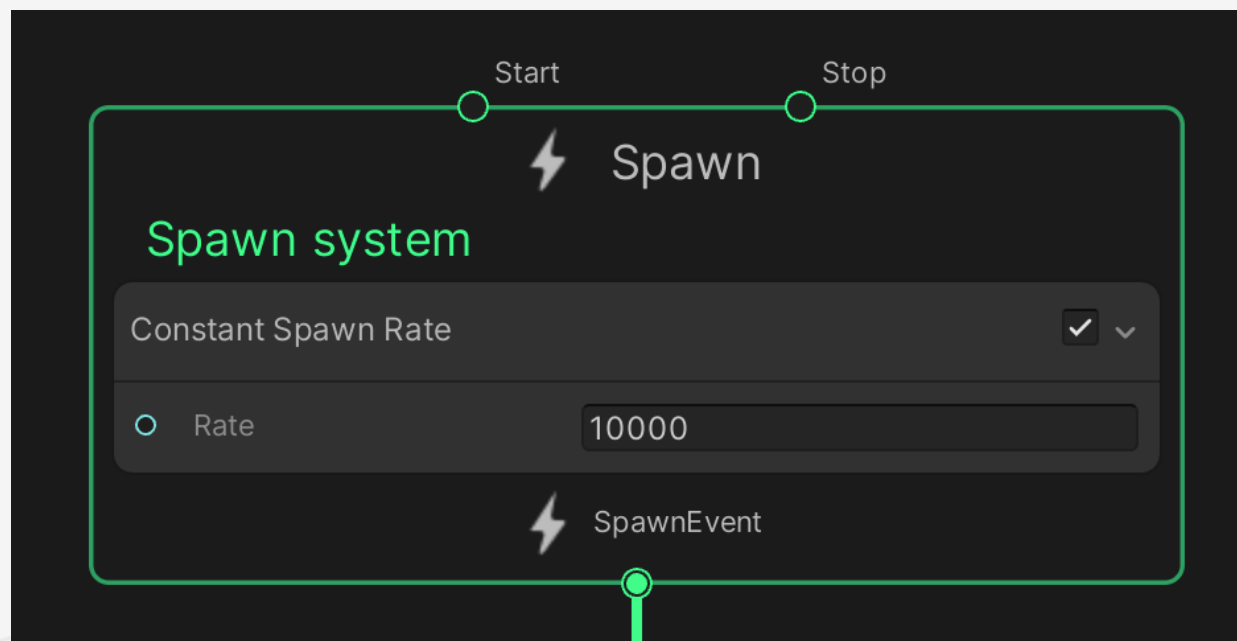
# Spawn Context

This first context controls the spawning of particles, similar to the Emission module for a Particle System. To quickly frame the Spawn context in view, select it and press F.

By default, a new VFX Graph uses a constant spawn rate, but you can set up variable, periodic, or bursts of particles, just like you did with the smoke burst effect.

1. Adjust the **Rate** property so that the VFX Graph version of the effect matches your original Particle System effect.

You can increase the spawn rate to a very high value without much of an impact on frame rate.

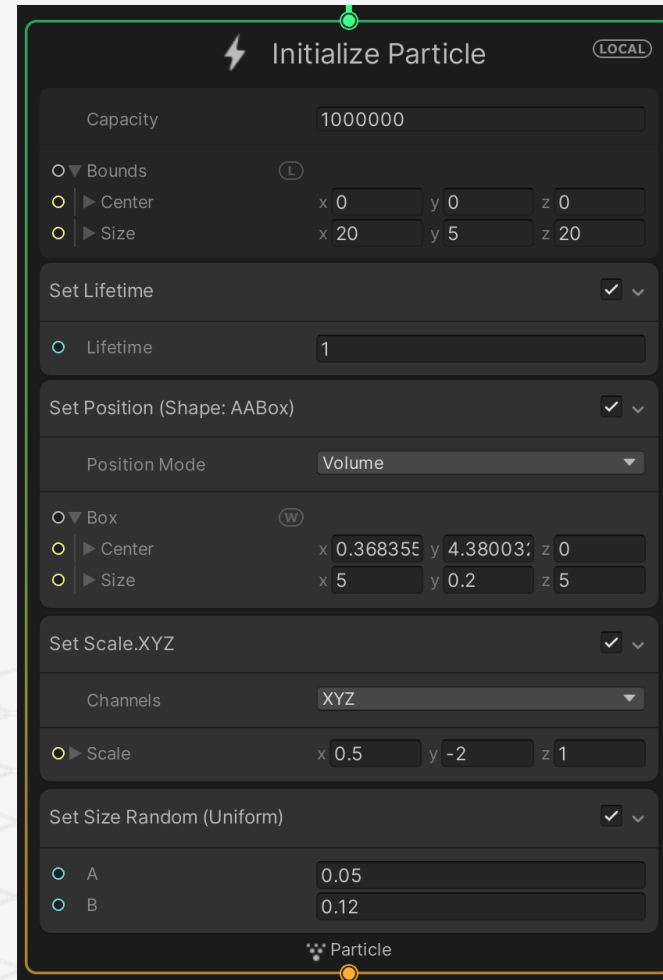


# Initialize Particle Context

The Initialize Particle context controls where and how the particles first appear.

This includes many of the same properties as the Particle System's main module, including the particles' initial size, shape, velocity, lifetime, and maximum capacity.

It also includes the equivalent of the **Shape** module, which defines the volume from which the particles are generated.



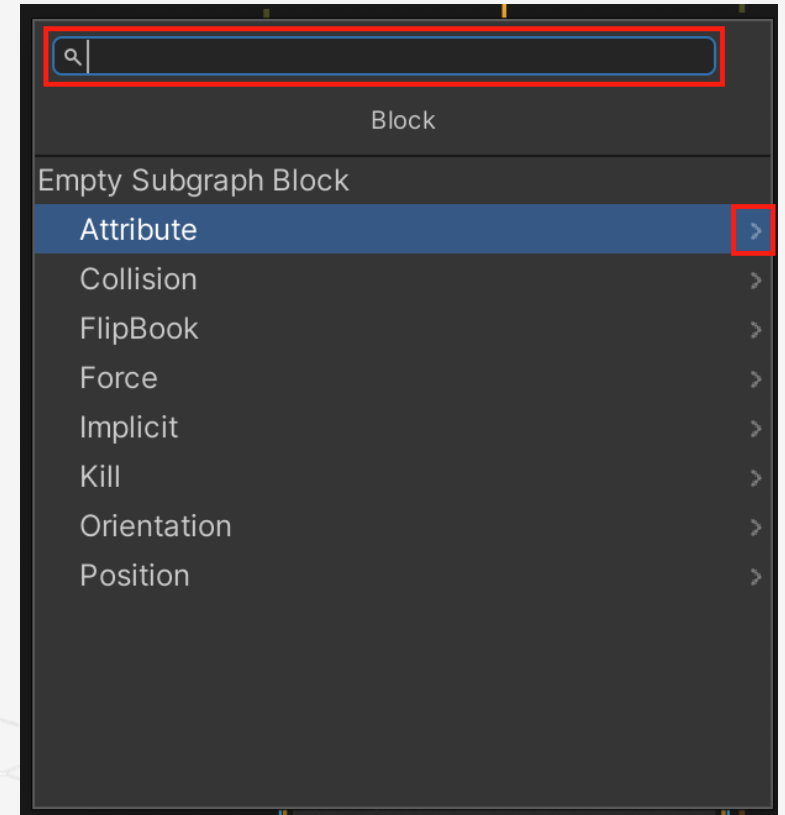
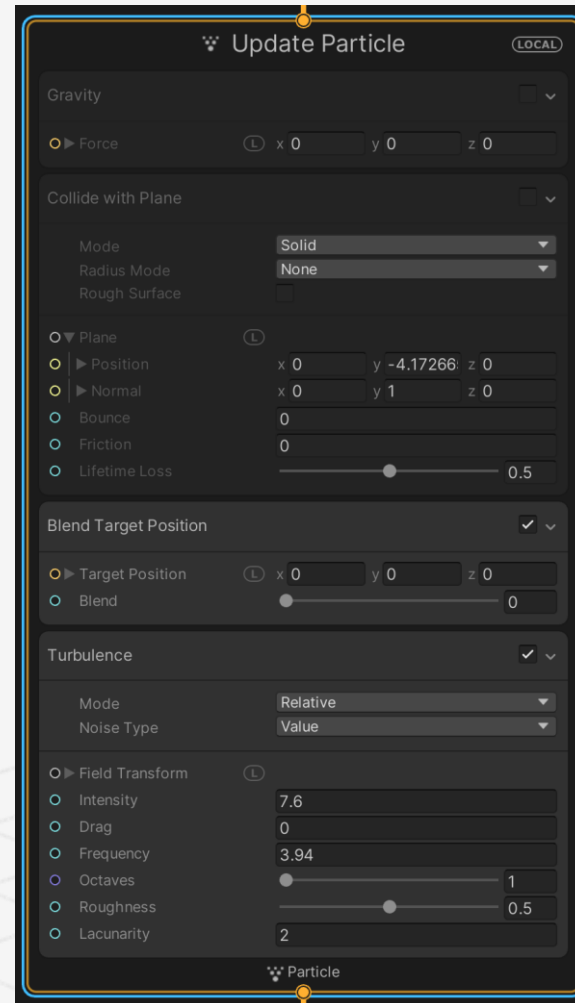
# Update Particle Context

The Update Particle context controls any changes to the particles from the moment they are generated until the moment they die (disappear).

This updates the particles and runs each frame. This includes properties similar to the Particle System's Color over Lifetime, Size over Lifetime, Noise (called Turbulence in VFX Graph), and Texture Sheet Animation (called Flipbook Player in VFX Graph).

Each of these behaviors can be added as a new **Block** in the context. In the screenshot above, for example, **Turbulence** is a Block.

3. Experiment with new Blocks by right-clicking in the Context and selecting **Create Block**. Then either browse through the sub-menus or use the search bar to explore.



# Output Particle Context

The Output Particle context controls the appearance of each of the particles, very similar to the **Renderer** module of a Particle System. This is where you assign a material for your particles.

